

## LABSHEET 8

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.metrics import (
    confusion_matrix,
    accuracy_score,
    precision_score,
    recall_score,
    f1_score
)
```

```
from google.colab import files
```

```
uploaded = files.upload()
```

Choose Files WA\_Fn-Us...-Attrition.csv

**WA\_Fn-UseC\_-HR-Employee-Attrition.csv**(text/csv) - 227977 bytes, last modified: 9/12/2026 - 100% done  
Saving WA\_Fn-UseC\_-HR-Employee-Attrition.csv to WA\_Fn-UseC\_-HR-Employee-Attrition.csv

```
# Load dataset
df = pd.read_csv("WA_Fn-UseC_-HR-Employee-Attrition.csv")

print(df.head())
print("\nDataset Shape:", df.shape)
print("\nColumns:")
print(df.columns)
```

|   | Age | Attrition | BusinessTravel    | DailyRate | Department             |  |
|---|-----|-----------|-------------------|-----------|------------------------|--|
| 0 | 41  | Yes       | Travel_Rarely     | 1102      | Sales                  |  |
| 1 | 49  | No        | Travel_Frequently | 279       | Research & Development |  |
| 2 | 37  | Yes       | Travel_Rarely     | 1373      | Research & Development |  |
| 3 | 33  | No        | Travel_Frequently | 1392      | Research & Development |  |
| 4 | 27  | No        | Travel_Rarely     | 591       | Research & Development |  |

|   | DistanceFromHome | Education | EducationField | EmployeeCount | EmployeeNumber |  |
|---|------------------|-----------|----------------|---------------|----------------|--|
| 0 | 1                | 2         | Life Sciences  | 1             | 1              |  |
| 1 | 8                | 1         | Life Sciences  | 1             | 2              |  |
| 2 | 2                | 2         | Other          | 1             | 4              |  |
| 3 | 3                | 4         | Life Sciences  | 1             | 5              |  |
| 4 | 2                | 1         | Medical        | 1             | 7              |  |

|   | RelationshipSatisfaction | StandardHours | StockOptionLevel |   |
|---|--------------------------|---------------|------------------|---|
| 0 | ...                      | 1             | 80               | 0 |
| 1 | ...                      | 4             | 80               | 1 |
| 2 | ...                      | 2             | 80               | 0 |
| 3 | ...                      | 3             | 80               | 0 |
| 4 | ...                      | 4             | 80               | 1 |

|   | TotalWorkingYears | TrainingTimesLastYear | WorkLifeBalance | YearsAtCompany |  |
|---|-------------------|-----------------------|-----------------|----------------|--|
| 0 | 8                 | 0                     | 1               | 6              |  |
| 1 | 10                | 3                     | 3               | 10             |  |
| 2 | 7                 | 3                     | 3               | 0              |  |
| 3 | 8                 | 3                     | 3               | 8              |  |
| 4 | 6                 | 3                     | 3               | 2              |  |

|   | YearsInCurrentRole | YearsSinceLastPromotion | YearsWithCurrManager |  |
|---|--------------------|-------------------------|----------------------|--|
| 0 | 4                  | 0                       | 5                    |  |
| 1 | 7                  | 1                       | 7                    |  |
| 2 | 0                  | 0                       | 0                    |  |
| 3 | 7                  | 3                       | 0                    |  |
| 4 | 2                  | 2                       | 2                    |  |

[5 rows x 35 columns]

Dataset Shape: (1470, 35)

Columns:

Index(['Age', 'Attrition', 'BusinessTravel', 'DailyRate', 'Department', 'DistanceFromHome', 'Education', 'EducationField', 'EmployeeCount', 'EmployeeNumber', 'EnvironmentSatisfaction', 'Gender', 'HourlyRate',

```
'JobInvolvement', 'JobLevel', 'JobRole', 'JobSatisfaction',
'MaritalStatus', 'MonthlyIncome', 'MonthlyRate', 'NumCompaniesWorked',
'Over18', 'OverTime', 'PercentSalaryHike', 'PerformanceRating',
'RelationshipSatisfaction', 'StandardHours', 'StockOptionLevel',
'TotalWorkingYears', 'TrainingTimesLastYear', 'WorkLifeBalance',
'YearsAtCompany', 'YearsInCurrentRole', 'YearsSinceLastPromotion',
'YearsWithCurrManager'],
dtype='object')
```

## Data Preprocessing

```
# Check missing values
print("Missing Values:")
print(df.isnull().sum())

# Remove duplicate rows
df = df.drop_duplicates()

# Convert categorical variables into numerical values
df_encoded = pd.get_dummies(df, drop_first=True)

print("\nEncoded Dataset:")
print(df_encoded.head())

print("\nNew Shape:", df_encoded.shape)
```

Missing Values:

|                          |   |
|--------------------------|---|
| Age                      | 0 |
| Attrition                | 0 |
| BusinessTravel           | 0 |
| DailyRate                | 0 |
| Department               | 0 |
| DistanceFromHome         | 0 |
| Education                | 0 |
| EducationField           | 0 |
| EmployeeCount            | 0 |
| EmployeeNumber           | 0 |
| EnvironmentSatisfaction  | 0 |
| Gender                   | 0 |
| HourlyRate               | 0 |
| JobInvolvement           | 0 |
| JobLevel                 | 0 |
| JobRole                  | 0 |
| JobSatisfaction          | 0 |
| MaritalStatus            | 0 |
| MonthlyIncome            | 0 |
| MonthlyRate              | 0 |
| NumCompaniesWorked       | 0 |
| Over18                   | 0 |
| OverTime                 | 0 |
| PercentSalaryHike        | 0 |
| PerformanceRating        | 0 |
| RelationshipSatisfaction | 0 |
| StandardHours            | 0 |
| StockOptionLevel         | 0 |
| TotalWorkingYears        | 0 |
| TrainingTimesLastYear    | 0 |
| WorkLifeBalance          | 0 |
| YearsAtCompany           | 0 |
| YearsInCurrentRole       | 0 |
| YearsSinceLastPromotion  | 0 |
| YearsWithCurrManager     | 0 |

dtype: int64

Encoded Dataset:

|   | Age | DailyRate | DistanceFromHome | Education | EmployeeCount | EmployeeNumber | \ |
|---|-----|-----------|------------------|-----------|---------------|----------------|---|
| 0 | 41  | 1102      | 1                | 2         | 1             | 1              |   |
| 1 | 49  | 279       | 8                | 1         | 1             | 2              |   |
| 2 | 37  | 1373      | 2                | 2         | 1             | 4              |   |
| 3 | 33  | 1392      | 3                | 4         | 1             | 5              |   |
| 4 | 27  | 591       | 2                | 1         | 1             | 7              |   |

|   | EnvironmentSatisfaction | HourlyRate | JobInvolvement | JobLevel | ... | \ |
|---|-------------------------|------------|----------------|----------|-----|---|
| 0 | 2                       | 94         | 3              | 2        | ... |   |
| 1 | 3                       | 61         | 2              | 2        | ... |   |
| 2 | 4                       | 92         | 2              | 1        | ... |   |
| 3 | 4                       | 56         | 3              | 1        | ... |   |
| 4 | 1                       | 40         | 3              | 1        | ... |   |

|   | JobRole_Laboratory Technician | JobRole_Manager | \ |
|---|-------------------------------|-----------------|---|
| 0 | False                         | False           |   |

|   |       |       |
|---|-------|-------|
| 1 | False | False |
| 2 | True  | False |

## Select Features + 80:20 Split

```
# Target variable
y = df_encoded["Attrition_Yes"]

# Features
X = df_encoded.drop("Attrition_Yes", axis=1)

# Split dataset into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42,
    stratify=y
)

print("Training data:", X_train.shape)
print("Testing data:", X_test.shape)
```

Training data: (1176, 47)  
Testing data: (294, 47)

```
# Create Decision Tree Classifier
model = DecisionTreeClassifier(
    criterion="gini",
    max_depth=5,
    random_state=42
)

# Train model
model.fit(X_train, y_train)

print("Decision Tree model trained successfully!")
```

Decision Tree model trained successfully!

## Train Decision Tree

```
y_pred = model.predict(X_test)

print("Actual values:")
print(y_test.values[:10])

print("\nPredicted values:")
print(y_pred[:10])
```

Actual values:  
[False False False False True False False False False False]

Predicted values:  
[False False False False False False False False False True]

## Confusion Matrix + Metrics

```
# Confusion Matrix
cm = confusion_matrix(y_test, y_pred)

print("Confusion Matrix:")
print(cm)

# Performance metrics
accuracy = accuracy_score(y_test, y_pred)
precision = precision_score(y_test, y_pred)
recall = recall_score(y_test, y_pred)
f1 = f1_score(y_test, y_pred)

print("\nModel Performance")
print("-----")
print("Accuracy :", round(accuracy, 4))
```

```
print("Precision:", round(precision, 4))
print("Recall  :", round(recall, 4))
print("F1 Score :", round(f1, 4))
```

Confusion Matrix:

```
[[236  11]
 [ 38   9]]
```

Model Performance

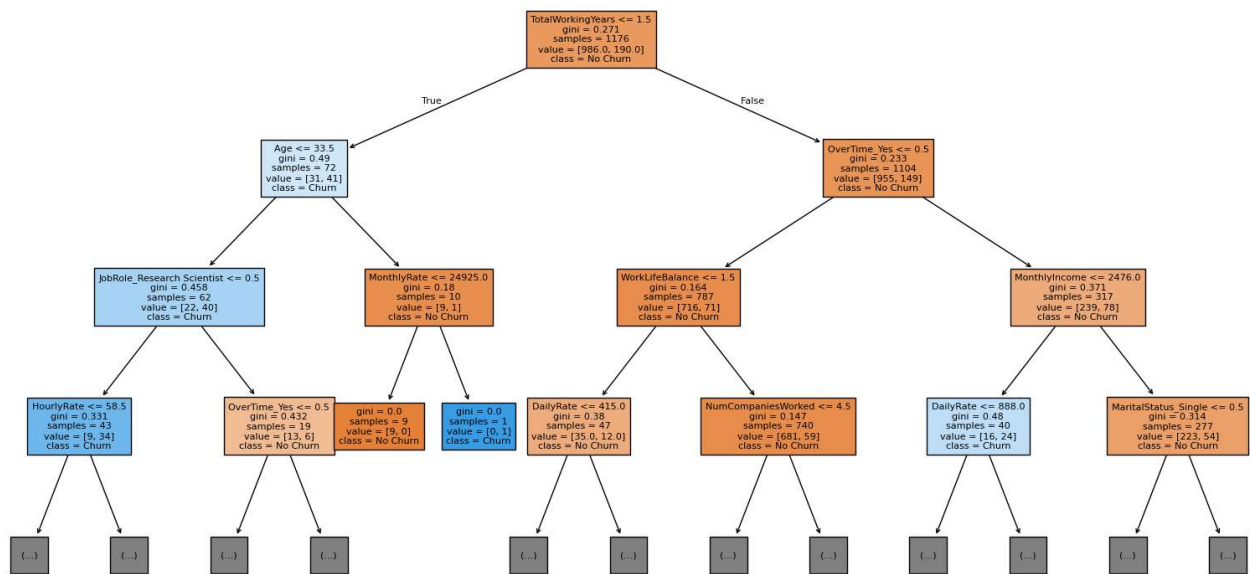
```
-----
Accuracy : 0.8333
Precision: 0.45
Recall   : 0.1915
F1 Score : 0.2687
```

```
plt.figure(figsize=(20, 10))

plot_tree(
    model,
    feature_names=X.columns,
    class_names=["No Churn", "Churn"],
    filled=True,
    max_depth=3,
    fontsize=8
)

plt.title("Decision Tree for Customer Churn Prediction")
plt.show()
```

Decision Tree for Customer Churn Prediction



Visualize Decision Tree

```
feature_importance = pd.DataFrame({
    "Feature": X.columns,
    "Importance": model.feature_importances_
})

feature_importance = feature_importance.sort_values(
    by="Importance",
```

```

        ascending=False
    )

    print("Most Important Features:")
    print(feature_importance.head(10))

```

```

Most Important Features:
      Feature  Importance
19  TotalWorkingYears    0.227763
46      OverTime_Yes    0.147191
  1      DailyRate    0.113395
  0           Age    0.100578
11    MonthlyIncome    0.096215
45  MaritalStatus_Single    0.057004
41  JobRole_Research Scientist    0.049864
13    NumCompaniesWorked    0.042111
  7      HourlyRate    0.038135
23    YearsInCurrentRole    0.033556

```

### Important Features

```

plt.figure(figsize=(10, 6))

top_features = feature_importance.head(10)

plt.barh(
    top_features["Feature"],
    top_features["Importance"]
)

plt.xlabel("Importance")
plt.ylabel("Feature")
plt.title("Top Features Influencing Customer Churn")
plt.gca().invert_yaxis()

plt.show()

```

